

Document Upload → RAG / Vector Search Pipeline

Step-by-step notes based on DocumentService, DocumentEmbeddingService, and PdfDocumentReader

1. Big Picture

Main flow: User uploads a PDF → physical file is saved → document metadata is saved in the database → PDF text is extracted → text is split into chunks → chunks are converted into embeddings by the VectorStore/EmbeddingModel → vectors are stored in PGVector → later, similarity search retrieves relevant chunks.

Security flow: The current user's ID is obtained from the authenticated security context. It is stored with the document and with vector metadata so document APIs and similarity search can be restricted to the current user.

Step	Component	Responsibility
1	DocumentService	Controls upload/delete/read workflow and ownership checks
2	FileUploadService	Saves and locates the physical file
3	DocumentRepository	Stores document metadata
4	DocumentReaderFactory	Chooses reader based on file extension
5	PdfDocumentReader	Extracts PDF text page by page
6	TextChunkingService	Splits extracted text into overlapping chunks
7	VectorStore / EmbeddingModel	Creates embeddings and stores vectors
8	PGVector	Persists vectors and metadata for similarity search

2. Step 1 — User Uploads a Document

The controller calls:

```
uploadDocument(MultipartFile file)
```

Spring's **MultipartFile** represents the uploaded file. Example: **HR-Policy.pdf**.

3. Step 2 — Identify the Logged-in User

The service executes:

```
Long userId = securityUtils.getCurrentUserId();
```

The user ID should come from the authenticated JWT/security context rather than from a request parameter.

Example: current user = **101**. This prevents a client from simply submitting another user's ID.

4. Step 3 — Get File Extension

For **HR-Policy.pdf**, `getFileExtension(...)` returns **pdf**. The extension is later used by DocumentReaderFactory to choose the correct reader.

5. Step 4 — Save the Physical File

The code calls:

```
String savedFilename = fileUploadService.saveFile(file);
```

This saves the actual PDF somewhere on the application's file system/storage. The database does not necessarily contain the PDF bytes; it stores metadata about the document.

6. Step 5 — Create and Save Document Metadata

DocumentEntity is built with userId, original filename, extension, file size, status, and timestamps:

```
DocumentEntity.builder()  
  .userId(userId)  
  .fileName(file.getOriginalFilename())  
  .fileType(fileExtension)  
  .fileSize(file.getSize())  
  .status(DocumentStatus.UPLOADED)  
  .createdAt(LocalDateTime.now())  
  .updatedAt(LocalDateTime.now())  
  .build();
```

Then documentRepository.save(document) inserts the metadata into the database and gives the document its database ID, for example **documentId = 50**.

7. Step 6 — Start the AI Processing Pipeline

The saved filename is converted into a Path and passed to:

```
embeddingService.processAndIndexDocument(document, filePath, fileExtension);
```

This starts the pipeline: **Extract** → **Chunk** → **Embed** → **Store**.

8. Step 7 — DocumentReaderFactory Chooses a Reader

The embedding service calls `readerFactory.getReader(fileExtension)`. For **pdf**, the factory returns **PdfDocumentReader**.

This design makes it easy to add other formats later, for example: pdf → PdfDocumentReader, docx → DocxDocumentReader, txt → TxtDocumentReader.

9. Step 8 — PdfDocumentReader Extracts Text

PDFBox opens the file:

```
PDDocument document = Loader.loadPDF(file);
int pageCount = document.getNumberOfPages();
```

The reader loops through every page. PDFTextStripper is configured for one page at a time and extracts the page text.

```
for (int pageNum = 1; pageNum <= pageCount; pageNum++) {
    stripper.setStartPage(pageNum);
    stripper.setEndPage(pageNum);
    String pageText = stripper.getText(document);
}
```

Empty pages are skipped. For every non-empty page, metadata such as `pageNumber`, `totalPages`, and `source=pdf` is added.

10. Step 9 — Add Security and Document Metadata

For each extracted page/section, the service starts with the reader's metadata and adds:

```
documentId = document.getId()
userId = document.getUserId()
fileName = document.getFileName()
fileType = document.getFileType()
```

Example final metadata for a chunk from page 5:

```
{ documentId: 50, userId: 101, fileName: "HR-Policy.pdf", fileType: "pdf", pageNumber: 5,
totalPages: 10, source: "pdf" }
```

Why this matters: metadata lets you identify which document/page a chunk came from and, importantly, lets similarity search apply a user-specific filter.

11. Step 10 — Chunk the Text

The code uses `CHUNK_SIZE = 1000` and `CHUNK_OVERLAP = 100`. Long text is divided into smaller pieces so the embedding model and vector search can work with focused pieces of content.

```
Conceptually:
Chunk 1 = characters 0-1000
Chunk 2 = characters 900-1900
Chunk 3 = characters 1800-2800
```

The 100-character overlap preserves context between neighboring chunks and reduces the chance that a sentence or important idea is split in a way that harms retrieval.

12. Step 11 — Create Spring AI Documents

Each `ChunkedContent` becomes a Spring AI Document containing two main things: **content** and **metadata**.

```
Document doc = new Document(chunk.getContent(), chunk.getMetadata());
```

13. Step 12 — Store in VectorStore

Finally:

```
vectorStore.add(documentsToStore);
```

The VectorStore works with the configured EmbeddingModel to turn document text into numerical vectors (embeddings), and the configured vector-store implementation persists those vectors and metadata. With PGVector, the vectors are stored in PostgreSQL's vector-capable storage.

14. What Is an Embedding?

An embedding is a numerical representation of the semantic meaning of text. For example, the text *Employees can take 20 days of annual leave* is transformed into a vector such as [0.12, -0.41, 0.87, ...]. A user's question is also embedded, and vector similarity is used to find chunks with similar meaning.

15. Step 13 — Mark the Document as INDEXED

If processing succeeds, DocumentService executes:

```
document.setStatus(DocumentStatus.INDEXED);  
document.setUpdatedAt(LocalDateTime.now());  
documentRepository.save(document);
```

The document's lifecycle is therefore roughly:

UPLOADED → processing → INDEXED

If processing fails:

UPLOADED → processing → FAILED

16. Failure Handling

The inner try/catch changes the document status to FAILED if extraction, chunking, embedding, or vector storage fails. The exception is then re-thrown and eventually wrapped as `ServiceProvisioningException` by the outer catch.

Important cleanup consideration: if the physical file is saved but database processing fails, an orphaned physical file may remain. A robust implementation should consider cleanup/transaction strategy.

17. Reading / Listing Documents Securely

`getDocument(documentId)` does:

```
Long userId = securityUtils.getCurrentUserId();
documentRepository.findByIdAndUserId(documentId, userId);
```

This is object-level authorization. If document 50 belongs to user 101 and user 102 requests document 50, the repository query should look for both ID=50 and user_id=102. No row is returned, so access is denied.

`getMyDocuments()` similarly calls `findAllByUserId(userId)`, so the list is restricted to the authenticated user.

18. Similarity Search Security

Document ownership checks are not enough for RAG. Similarity search must also be restricted to the current user's vectors.

```
Unsafe concept:
vectorStore.similaritySearch(question);
```

Safer concept:

get current userId → create a similarity-search filter on userId → search only that user's chunks.

For example, if user 101 asks a question, the vector search should retrieve chunks whose metadata contains `userId=101`. This prevents user 101 from receiving semantically similar chunks belonging to user 102.

19. Delete Flow

The intended delete flow is: authenticate user → find document by documentId AND userId → delete vector embeddings → delete database metadata → delete physical file.

Current code issue: `deleteDocumentEmbeddings(documentId)` only logs a message; it does not actually delete vectors. Also, the shown `deleteDocument()` does not call a physical-file deletion method. These should be addressed in a production implementation.

20. Complete Example

Suppose user 101 uploads HR-Policy.pdf with 10 pages:

```
User 101
↓
DocumentService
↓
Save HR-Policy.pdf
↓
DB: documentId=50, userId=101, status=UPLOADED
↓
DocumentEmbeddingService
↓
PdfDocumentReader → extract 10 pages
↓
TextChunkingService → create chunks
↓
Add documentId/userId/page metadata
↓
```

VectorStore + EmbeddingModel
↓
PGVector stores vectors + metadata
↓
DB status = INDEXED

21. Responsibilities of Each Class

Class	Main responsibility
DocumentService	Business workflow, ownership/security checks, status updates, delete/read/list operations
DocumentEmbeddingService	AI pipeline: extraction, chunking, creation of Spring AI Documents, vector indexing
PdfDocumentReader	PDF-only text extraction using Apache PDFBox
DocumentReaderFactory	Selects the correct reader for the file extension
TextChunkingService	Splits text into overlapping chunks
EmbeddingModel	Converts text into embeddings; used by the configured vector-store flow
VectorStore	Adds/searches vector documents and persists vectors through the configured implementation
DocumentRepository	Persists relational document metadata
SecurityUtils	Provides the authenticated user's ID

22. Interview / Assignment Summary

Remember this one-line explanation: DocumentService manages the secure document lifecycle; PdfDocumentReader extracts text page-by-page; TextChunkingService breaks text into overlapping chunks; Spring AI's VectorStore/EmbeddingModel converts chunks into embeddings and stores them in PGVector with document and user metadata; similarity search later retrieves relevant chunks, but must apply a userId filter to prevent cross-user data access.

Key security rule

Always enforce ownership in the database query for document APIs, and enforce userId filtering in vector similarity search. Never trust a userId supplied by the client.